# Machine Learning Foundations

## Week 1: TA session(Tuesday)

Prayag Buch

October 2026

# Today's Roadmap

1. **Can You Code Everything?**
2. **The Language of Learning**
3. **What is Learning?**
4. **Supervised Learning**
5. **Unsupervised Learning**
6. **Density Estimation**

# Part 1: Can You Code Everything?

# Simple patterns like a checkerboard need just 5 lines of Python

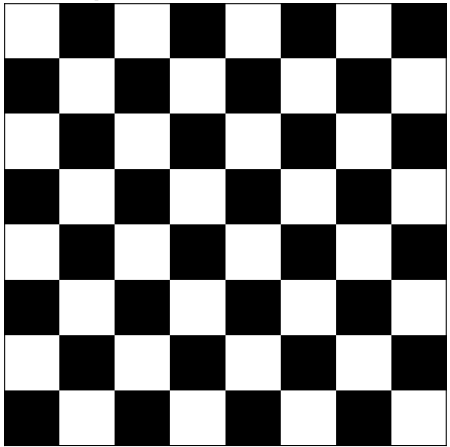
**What we know:**

We can write explicit instructions (algorithms) to generate simple, rigid structures.

```
import numpy as np
import matplotlib.pyplot as plt

board = np.indices((8,8)).sum(axis=0) %
plt.imshow(board, cmap='gray')
```

**A Simple Pattern: 5 Lines of Code**



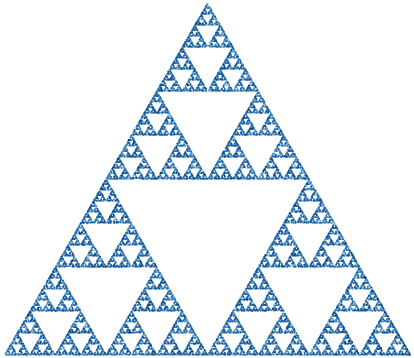
# Even a bit more complex patterns can be generated with loops and math

As complexity increases, we can still use mathematical recursion and deterministic rules.

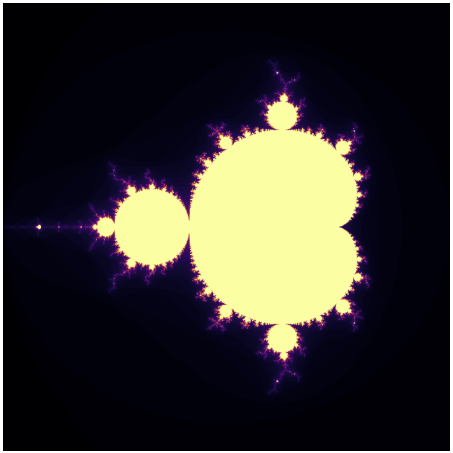
**The Sierpinski Triangle** is infinitely complex but is perfectly programmable.

*Every pixel follows a rule we can write.*

Sierpinski Triangle



# More Complex? - Add colour



**The limit of manual coding:**

Every pixel in this Mandelbrot set follows an explicit formula: $z_{n+1} = z_n^2 + c$.

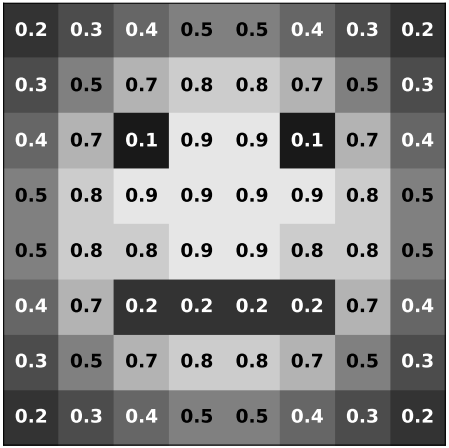
Beautiful, but entirely deterministic. We must know the exact relation in advance.

# Challenge: Can you write a program that generates a human face?

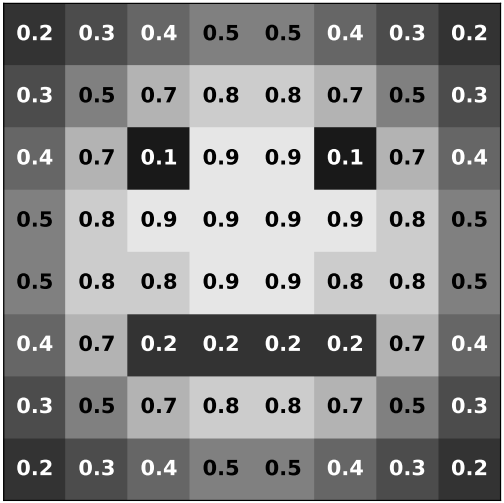
**Think about it:**

Can you write `if-else` rules for what makes a face look like a face?

*Impossible!*



# No finite set of if-else rules can capture natural image structure



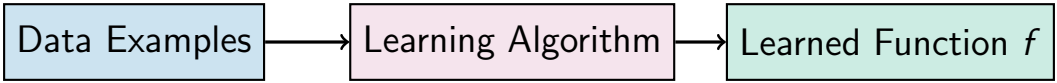

**Astronomical complexity:**

$256 \times 256$ pixels $\times$ 3 channels $= 196{,}608$ numbers.

The number of possible images is $256^{196608}$. Only a vanishingly tiny fraction of these "look like faces".

# Instead of writing rules, we show examples and let the machine learn

**Machine Learning = Teaching machines to find patterns we can't code**



# Poll: Can Code Replace ML?

**POLL Think carefully!**

A programmer writes 10 million lines of if-else rules to detect spam emails. Is this Machine Learning?

A) Yes, the program learns which emails are spam

B) Yes, 10 million rules can capture any pattern

C) No, the rules were hand-written, not learned from data

D) It depends on whether the rules are accurate

# Poll Answer

**Answer**

**C) No, the rules were hand-written, not learned from data**

# The Language of Learning

Data, Vectors, and Models

# Numerical Representataion

To make machines learn, we must speak their language, i.e. Numbers!

- **Images** $\rightarrow$ Pixel arrays (RGB values)
- **Text** $\rightarrow$ Token IDs and embeddings
- **Audio** $\rightarrow$ Waveform amplitudes

*Even GPT doesn't see "Hello world".*

It sees: `[15496, 995]`

Try it:
`https://platform.openai.com/tokenizer`

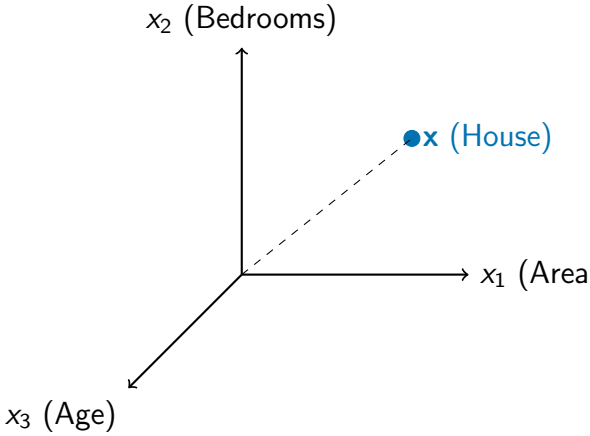
# Each data sample is a vector $\mathbf{x} \in \mathbb{R}^n$ a point in $n$-dimensional space

**Data as Vectors:**

A single house can be described by $n$ features:

$$\mathbf{x} = \begin{bmatrix} \text{Area} \\ \text{Bedrooms} \\ \text{Age} \\ \text{Distance to Metro} \end{bmatrix} \in \mathbb{R}^4$$

*Every house is a point in 4D space!*



# PYQ: Vector Relationships

**Previous Year Question**

Given vectors: $\mathbf{v}_1 = (1, 2, 3)^T$, $\mathbf{v}_1 = -2\mathbf{v}_2$, and $\mathbf{v}_3 = (1, -5, 3)^T$.
Which of the following statements are true?

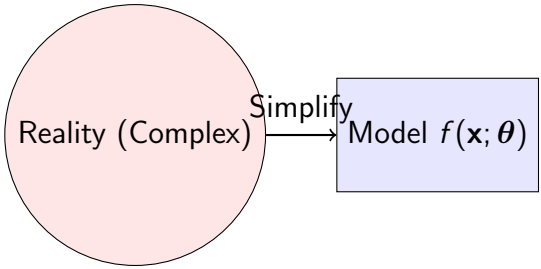
1. A) $\mathbf{v}_1$ and $\mathbf{v}_2$ are parallel
2. B) $\mathbf{v}_1$ and $\mathbf{v}_2$ are perpendicular
3. C) $\mathbf{v}_2$ and $\mathbf{v}_3$ are perpendicular
4. D) $\mathbf{v}_1$ and $\mathbf{v}_3$ are perpendicular

# Model

**A model is a mathematical simplification of reality.**

We deliberately simplify to capture the essential structure.

A model with parameters $\boldsymbol{\theta}$ is written as $f(\mathbf{x}; \boldsymbol{\theta})$.



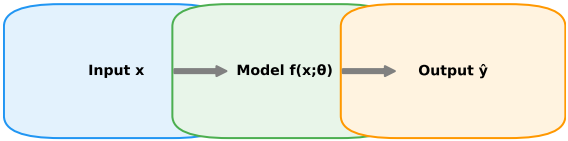
# A predictive model learns a mapping $f : X \to Y$ from data

Given input **x**, predict output $y$.

**Examples:**

- Spam filter
- House price prediction
- Medical diagnosis

*Like Google Maps predicting your ETA from [distance, traffic, time of day].*



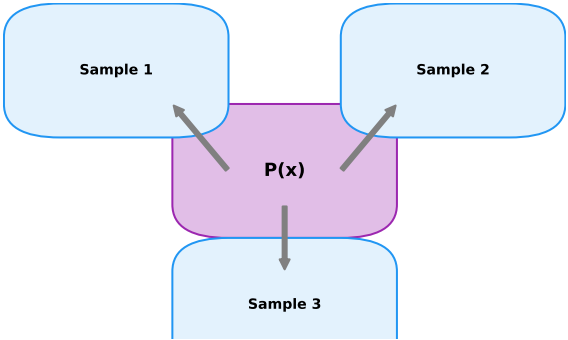
# A probabilistic model learns to generate new data

Instead of predicting $y$, we learn the underlying data distribution $P(\mathbf{x})$.

**Examples:**

- Image generation (GANs, Diffusion)
- Text generation (GPT)

*Like learning what "typical" music sounds like, then composing new songs.*



# Poll: The Perfect Model

**POLL**

You build a model that perfectly predicts every single point in your training data (zero error). Your friend's model has some training error. Who has the better model?

A) You, zero error is always best

B) Your friend, some error is actually a good sign

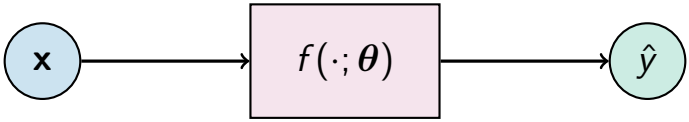
C) Impossible to tell without testing on new data

D) Whoever has fewer parameters (i.e less expensive)

# What is ‘Learning’ in ML?

# Given features $\mathbf{x}$, the model $f(\mathbf{x}; \boldsymbol{\theta})$ outputs predictions

The parameters $\boldsymbol{\theta}$ act as adjustable "knobs".




*Learning = finding the "best" parameters $\boldsymbol{\theta}$. But what does "best" mean?*

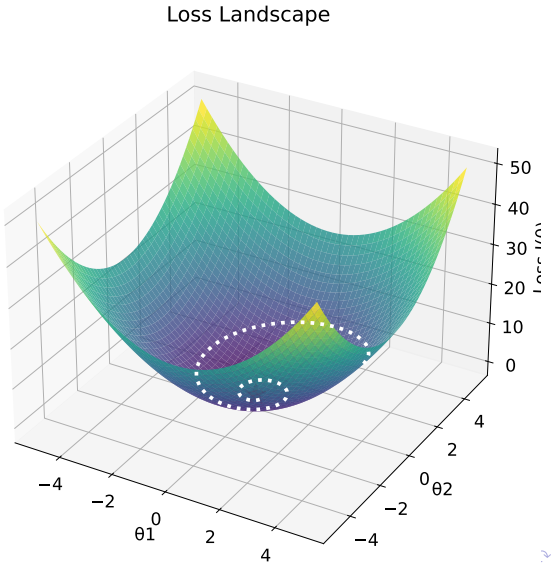
# The loss function $\mathcal{L}(\boldsymbol{\theta})$ measures how wrong our predictions are

We want to find the parameters that minimize the loss function:

$$\boldsymbol{\theta}^* = \arg\min \mathcal{L}(\boldsymbol{\theta})$$

Learning is an optimization problem: finding the bottom of the loss landscape valley!

*Like a ball rolling freely downhill; it naturally settles at the lowest point.*



## Previous Year Question

Which of the following statements are correct?

1. A) $\mathbb{I}(444 \pmod 3 = 0 \text{ and } \pi \in \mathbb{R}) = 1$
2. B) Mean Squared Error is a suitable loss function for classification
3. C) Encoder-decoder loss can be negative
4. D) Gaussian Mixture Models can be used to model multimodal data

# Poll: Training Loss Showdown

Model A has training loss $= 0.01$
Model B has training loss $= 0.50$

Which model will perform better on **new, unseen data**?

A) Model A , lower loss is always better
B) Model B, higher training loss means it generalizes better
C) Cannot determine from training loss alone
D) The one with more parameters

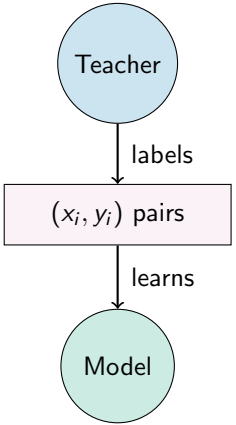
# Supervised Learning

# Supervised = Learning with a teacher who provides correct answers

For every input $\mathbf{x}$, we are given the correct output $y$.

**Dataset:**
$\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$

The "supervisor" provides ground truth labels.



# Two Main Types of Supervised Learning

**Classification**

- Predicting a **discrete category**
- Is this email spam? (Yes/No)
- What animal is in this image?
- $y \in \{0, 1, 2, \ldots, K\}$

**VS**

**Regression**

- Predicting a **continuous number**
- What is the price of this house?
- What is tomorrow's temperature?
- $y \in \mathbb{R}$

# PYQ: Identify the Task Type

**Previous Year Question**

Which of the following ML tasks require Classification?

1. A) Credit card fraud detection
2. B) Sentiment analysis (Positive/Negative/Neutral)
3. C) Predicting number of emails a user will receive
4. D) Predicting whether a patient has cancer

# PYQ Answer: Identify the Task Type

**Solution**

- **A) Fraud detection:** fraud / not-fraud $\Longrightarrow$ **Classification (Binary)**
- **B) Sentiment analysis:** 3 discrete categories $\Longrightarrow$ **Classification**
- **C) Number of emails:** count (continuous scale) $\Longrightarrow$ **Regression**
- **D) Cancer diagnosis:** cancer / no-cancer $\Longrightarrow$ **Classification (Binary)**

**Correct Options: A, B, D**

# PYQ: Bank Loan Scenario

**Previous Year Question**

A bank decides loan approval based on income, credit score, and past records. Which technique is this?

- A) Regression
- B) Classification
- C) Clustering
- D) Dimensionality Reduction

# In regression, $\mathbf{x} \in \mathbb{R}^n$ is a feature vector and $y \in \mathbb{R}$ is a target

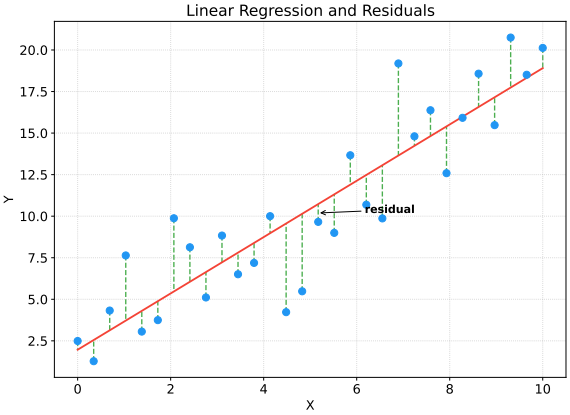



Example: Predict house price from [area, age, location]

# The squared loss penalizes predictions that are far from truth

The loss measures the total squared **vertical** distance:

$$\mathcal{L} = \sum_{i=1}^{n} \left(y_i - f(\mathbf{x}_i)\right)^2$$




Why squared?

- Differentiable everywhere
- Penalizes large errors heavily
- Has a unique minimum

# PYQ: Comparing Regression Models

**Previous Year Question**

Given dataset $D = \{(1, 1), (2, 4), (3, 7)\}$.
Consider two models:

$$f_1(x) = 2x - 1$$

$$f_2(x) = 3x - 2$$

Which model has the lower Mean Squared Error (MSE)?

# PYQ Answer: Comparing Regression Models

**Solution**

**Evaluate** $f_1(x) = 2x - 1$**:**

- $x = 1 \implies \hat{y} = 1$, error $= 0$ $\quad x = 2 \implies \hat{y} = 3$, error $= 1$
  $x = 3 \implies \hat{y} = 5$, error $= 2$

$\text{MSE}_1 = (0^2 + 1^2 + 2^2)/3 = 5/3$

**Evaluate** $f_2(x) = 3x - 2$**:**

- $x = 1 \implies \hat{y} = 1$, error $= 0$ $\quad x = 2 \implies \hat{y} = 4$, error $= 0$
  $x = 3 \implies \hat{y} = 7$, error $= 0$

$\text{MSE}_2 = 0$

**$f_2(x)$ has the lower (zero) MSE.**

# PYQ: MSE Computation

**Previous Year Question**

Model: $h(x_1, x_2) = (x_1 + x_2)/2$
Data points $(\mathbf{x}, y)$:

- $([1, 2], 2)$
- $([2, 3], 3)$
- $([3, 4], 4)$
- $([4, 5], 5)$

What is the Mean Squared Error?

# PYQ Answer: MSE Computation

**Solution**

Evaluate $h(x)$ for each point:

- $[1, 2] \implies h = 1.5 \implies (y - h)^2 = 0.25$
- $[2, 3] \implies h = 2.5 \implies (y - h)^2 = 0.25$
- $[3, 4] \implies h = 3.5 \implies (y - h)^2 = 0.25$
- $[4, 5] \implies h = 4.5 \implies (y - h)^2 = 0.25$

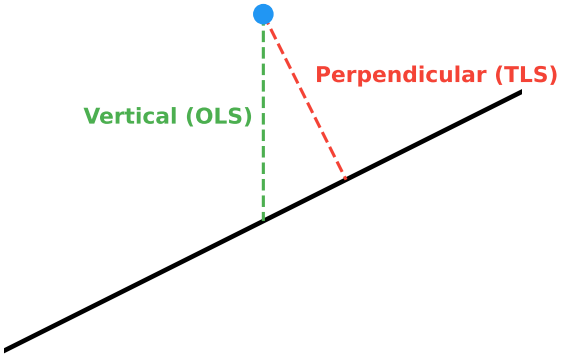
**MSE** $= (0.25 \times 4)/4 = \mathbf{0.25}$

# What does the squared loss actually measure?

**Quick Poll**

In linear regression, the loss computes the _____ distance from data points to the fitted line.

A) Perpendicular (shortest) distance
B) Vertical distance
C) Horizontal distance
D) Euclidean distance in feature space

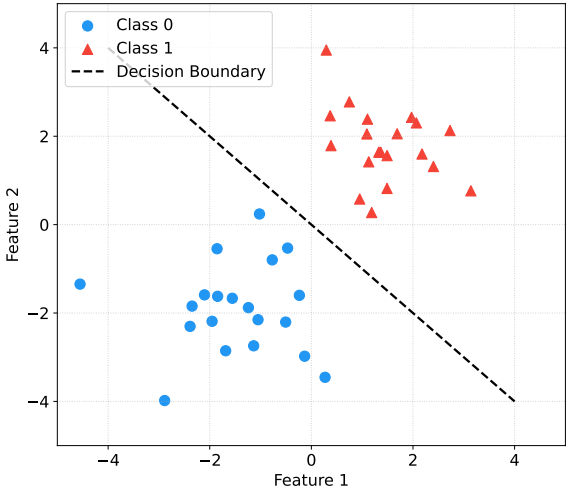


# In classification, the model predicts which class a point belongs to

Output is discrete: $y \in \{0, 1\}$ (Binary) or $y \in \{1, 2, \ldots, K\}$ (Multi-class).

The model learns a **decision boundary** separating classes.

*To be studied in detail in subsequent weeks*



# PYQ: Classification Loss

**Previous Year Question**

Model: $\hat{y} = u(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$

where $z = 0.5x_1 - x_2 + 0.4x_3$.

Data points $(\mathbf{x}, y)$:

- $([0, 1, 1], 0)$
- $([1, 0, 1], 1)$
- $([1, 0, 0], 1)$
- $([1, 1, 1], 1)$
- $([1, 1, 0], 1)$

What is the misclassification loss (fraction of wrong predictions)?

# PYQ Answer: Classification Loss

**Solution**

Evaluate $z$ and $u(z)$ for each point:

- $[0, 1, 1], y = 0 \implies z = -0.6 \implies u(z) = 0$ ✓
- $[1, 0, 1], y = 1 \implies z = 0.9 \implies u(z) = 1$ ✓
- $[1, 0, 0], y = 1 \implies z = 0.5 \implies u(z) = 1$ ✓
- $[1, 1, 1], y = 1 \implies z = -0.1 \implies u(z) = 0$ × **Wrong**
- $[1, 1, 0], y = 1 \implies z = -0.5 \implies u(z) = 0$ × **Wrong**

**Loss = 2/5 = 0.4 (40% misclassification)**

# PYQ: Least Misclassification (2D)

**Previous Year Question**

Dataset:
Positive (+1): $[0, 1], [0, 2], [1, 3]$
Negative (-1): $[3, 4], [4, 5], [7, 6]$
Which model has the least misclassification error?

- A) Sign$(x_1 - 2)$
- B) Sign$(x_2 + 2)$
- C) Sign$(x_1 + x_2)$
- D) Sign$(x_2 - x_1)$

# PYQ Answer: Least Misclassification (2D)

**Solution**

Evaluate Sign$(x_2 - x_1)$ (Option D):

- $[0,1] \implies 1 > 0 \implies +1$ ✓ $\quad [0,2] \implies 2 > 0 \implies +1$ ✓
  $[1,3] \implies 2 > 0 \implies +1$ ✓
- $[3,4] \implies 1 > 0 \implies +1$ × $\quad [4,5] \implies 1 > 0 \implies +1$ ×
  $[7,6] \implies -1 < 0 \implies -1$ ✓

Errors $= 2/6$. All other models have $\geq 3/6$ errors.

**Correct Option: D**

# PYQ: Perfect Classifier (3D)

**Previous Year Question**

Dataset:
Class -1: $[0, 1, 1], [0, 1, 0]$
Class $+1$: $[1, 1, 3], [1, 0, 5], [0, 0, 6], [1, 0, 4]$
Which model gives 0 errors? (Assume $\text{Sign}(0) = 1$)

- A) $\text{Sign}(x_1 + x_2 - 2)$
- B) $\text{Sign}(x_1 + x_2 + x_3)$
- C) $\text{Sign}(x_3 - x_2 - x_1)$
- D) $\text{Sign}(x_1 + x_2 + x_3 - 5)$

# PYQ Answer: Perfect Classifier (3D)

**Solution**

Check Option D: Sign$(x_1 + x_2 + x_3 - 5)$

- $[0,1,1] \implies 2-5=-3 \implies -1$ ✓ $[0,1,0] \implies 1-5=-4 \implies -1$ ✓
- $[1,1,3] \implies 5-5=0 \implies +1$ ✓ $[1,0,5] \implies 6-5=1 \implies +1$ ✓
- $[0,0,6] \implies 6-5=1 \implies +1$ ✓ $[1,0,4] \implies 5-5=0 \implies +1$ ✓

**Correct Option: D (0 errors)**

# Poll: Classification or Regression?

**POLL**

A model predicts stock prices rounded to the nearest decimal. The output is always a whole number like Rs. 142, Rs. 271, etc. Is this Classification or Regression?

A) Classification, the output is discrete (whole numbers)

B) Regression, the output is a continuous quantity

C) Both, it can be framed either way

D) Neither, stock prediction is unsupervised

# Training data teaches, validation data tunes, test data evaluates

- **Training Data** $\rightarrow$ The textbook (learn parameters)
- **Validation Data** $\rightarrow$ The practice exam (tune hyperparameters)
- **Test Data** $\rightarrow$ The final exam (evaluate once)

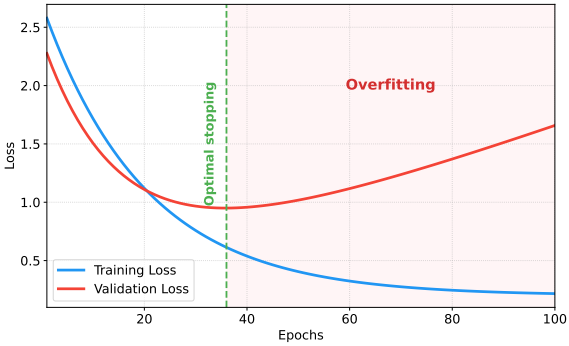
*If you study the final exam answers, you'll score 100% but learn nothing. Same with models!*

# When validation loss rises while training loss falls, the model is overfitting

**Overfitting:** The model memorizes training data but fails on new data.

**Sweet Spot:** We stop training at the minimum of the validation loss.

*Like a student who memorizes every practice question word-for-word but can't solve new problems.*



# Unsupervised Learning

# High-dimensional data is expensive, but redundancy means we can compress

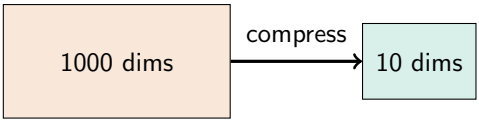
A 1000-dimensional feature vector means the model needs millions of parameters.

But what if only 10 dimensions truly matter?

*A 10-megapixel photo can be compressed to 100KB JPEG. Most of those pixels are redundant!*

**Goal of Dimensionality Reduction:**

Find the important directions in the data.



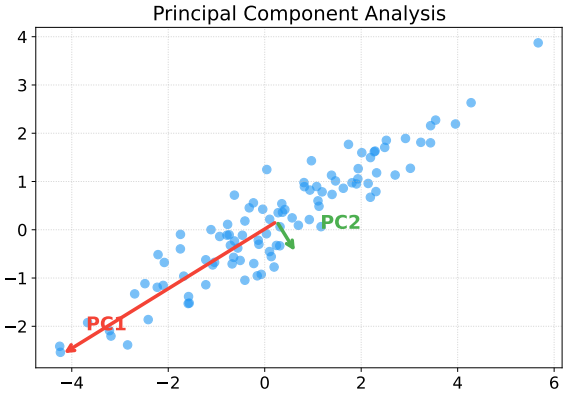
# Dimensionality reduction minimizes the reconstruction error

We want a compact representation that preserves the essential structure.

$$\min \left\| \mathbf{X} - \hat{\mathbf{X}} \right\|^2$$

Where $\hat{\mathbf{X}} = \text{decode}(\text{encode}(\mathbf{X}))$.

*How much information do we lose by compressing?*



# PYQ: Comparing Encoder-Decoder Pairs (2D $\rightarrow$ 1D)

**Previous Year Question**

Data: $(2, 4), (6, 8)$
Decoder for both pairs: $g(u) = [u, u]^T$
Which encoder is better?

- **Pair 1:** $f(\mathbf{x}) = (x_1 + x_2)/2$
- **Pair 2:** $f(\mathbf{x}) = x_1 - x_2$

# PYQ Answer: Comparing Encoder-Decoder Pairs

**Solution**

**Pair 1:** $f = (x_1 + x_2)/2$

- $(2,4) \implies u = 3 \implies \hat{\mathbf{x}} = (3,3)$. Error $= 1 + 1 = 2$.
- $(6,8) \implies u = 7 \implies \hat{\mathbf{x}} = (7,7)$. Error $= 1 + 1 = 2$.

Mean Loss $= 2$.

**Pair 2:** $f = x_1 - x_2$

- $(2,4) \implies u = -2 \implies \hat{\mathbf{x}} = (-2,-2)$. Error $= 16 + 36 = 52$.
- $(6,8) \implies u = -2 \implies \hat{\mathbf{x}} = (-2,-2)$. Error $= 64 + 100 = 164$.

Mean Loss $= 108$.

**Pair 1 is vastly better (loss = 2 vs 108).**

# PYQ: Reconstruction Loss (3D $\rightarrow$ 1D)

**Previous Year Question**

Encoder: $f(x_1, x_2, x_3) = 0.5x_1 + 0.5x_2 - x_3$
Decoder: $g(u) = [u, u, u]^T$
Data points:

- $(2, 4, 3)$
- $(5, 5, 6)$
- $(2, 2, 1)$
- $(4, 4, 6)$

What is the mean reconstruction loss?

# PYQ Answer: Reconstruction Loss

**Solution**

- $(2,4,3) \implies u = 0 \implies \hat{\mathbf{x}} = (0,0,0) \implies \text{Error} = 4 + 16 + 9 = 29$
- $(5,5,6) \implies u = -1 \implies \hat{\mathbf{x}} = (-1,-1,-1) \implies \text{Error} = 36 + 36 + 49 = 121$
- $(2,2,1) \implies u = 1 \implies \hat{\mathbf{x}} = (1,1,1) \implies \text{Error} = 1 + 1 + 0 = 2$
- $(4,4,6) \implies u = -2 \implies \hat{\mathbf{x}} = (-2,-2,-2) \implies \text{Error} = 36 + 36 + 64 = 136$

**Mean Reconstruction Loss** $= \frac{288}{4} = 72$

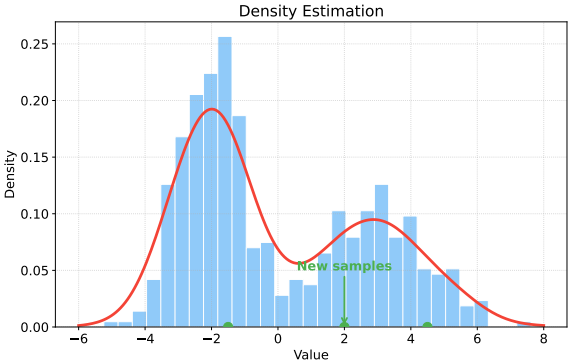
# Part 7: Density Estimation

# A generative model learns the probability distribution $P(\mathbf{x})$

**Goal:** Given training samples, learn $P(\mathbf{x})$ so we can:

- Generate new realistic samples
- Detect anomalies (low probability = suspicious)
- Understand data structure

*Like learning what "typical" handwriting looks like, so you can spot forgeries.*



# We find parameters that make the training data as probable as possible

**Maximum Likelihood Estimation:**
Maximize probability, or minimize Negative Log-Likelihood (NLL).

*"Find the distribution that would have been most likely to produce the data we actually saw."*

$$\boldsymbol{\theta}^* = \arg\min \underbrace{-\sum_{i=1}^{n} \log P(\mathbf{x}_i; \boldsymbol{\theta})}_{\text{Negative Log-Likelihood}}$$

Why log? Converts products to sums, easier to differentiate!

# Poll: Density Estimation for Classification?

**POLL**

Can density estimation (learning $P(\mathbf{x})$) be used to do classification?

A) No, density estimation is unsupervised, classification is supervised

B) No, density estimation learns $P(\mathbf{x})$, but classification needs $P(y|\mathbf{x})$

C) Yes, learn $P(\mathbf{x}|y)$ for each class, then use Bayes' theorem

D) Yes, but only for binary classification

# Poll Answer

**Answer**

## C) Yes, learn $P(\mathbf{x}|y)$ for each class, then use Bayes' theorem

**This is the foundation of Bayesian classification:**

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y) \cdot P(y)}{P(\mathbf{x})}$$

# Week 1 Summary

- **Machine Learning:** Approximating complex functions from data (not coding rules)
- **Data:** Numerical vectors; **Models:** Parameterized functions
- **Learning:** Finding parameters that minimize a loss function
- **Three Paradigms:**
  - **Supervised:** Regression (squared loss) & Classification
  - **Unsupervised:** Dimensionality reduction (reconstruction error)
  - **Generative:** Density estimation (NLL)

# See You Next Week!

Questions?